

FlexQ AWS セットアップ手順書【運営者向け】

この手順書は、アプリ **FlexQ** の運営者が **自分名義の AWS アカウント** を作成し、アプリのサーバー環境(検証用 staging / 本番用 production)を立ち上げて、現在開発者が管理しているデータを引き継ぐまでの手順をまとめたものです。

- **エンジニアの知識は前提にしていません。** 上から順に、書いてあるとおりに進めれば完了できます
- パソコンは **Mac** を前提としています
- 手順書の中に出てくる `flexq` は、アプリ名 FlexQ の AWS 上での表記(半角英小文字)です
- 詰まったら無理に進めず、開発者に画面共有で相談してください

この手順書でやること(全体の流れ)

第 1 章	必要なもの・ツールの準備	← Mac にツールを入れる
第 2 章	AWS アカウントの作成	← ブラウザで登録・安全設定
第 3 章	作業用ユーザーの作成	← ブラウザで IAM 設定
第 4 章	ターミナルと AWS をつなぐ	← ターミナルで接続設定
第 5 章	サーバー環境の立ち上げ	← コマンド 1 回で全部できる
第 6 章	メール送信の設定	← ブラウザで SES 設定
第 7 章	既存データの引っ越し	← コマンドでコピー
第 8 章	完了報告(開発者へ渡すもの)	
第 9 章	困ったときは	

作業時間の目安: 全体で 2~4 時間(待ち時間を除く)。1 日で終わらせる必要はなく、章の区切りで中断して構いません。

開発者とのやり取り(受け渡し)の全体像

作業を進める中で、開発者へ情報を渡すタイミングが 5 回あります。該当する章の終わりに  **マーク**で案内するので、その都度渡してください(第 8 章まで溜めないこと。特に第 5 章の `ApiUrl` は、渡した時点から開発者が並行して作業を進められます)。

順番	タイミング	開発者へ渡すもの
①	第 1 章の前	GitHub のユーザー名(リポジトリに招待してもらうため)
②	第 3 章の終わり	<code>flexq-deploy</code> のアクセスキー(ID とシークレット)
③	第 5 章の終わり	staging / production の <code>ApiUrl</code> (URL 2 つ)
④	第 6 章(承認メール到着時)	SES の検証完了と本番アクセス承認の連絡
⑤	第 7 章の終わり	データ検証結果の報告

逆に、開発者から**受け取る**もの(キー・シークレット類)は 1-2 にまとめてあり、第 1 章を始める前にもらっておきます。

 キーやシークレットを渡すときは、必ずパスワードマネージャーの共有機能など安全な方法で。メール・LINE での平文送信は厳禁です(以降の章でも同じ)。

第 1 章 必要なもの・ツールの準備

1-1. 事前に用意するもの

用意するもの	用途
メールアドレス	AWS アカウントの登録用(今後も運営者が管理し続けられるもの)
クレジットカード	AWS の支払い登録用(現在の利用規模なら月数十円～数百円程度)
SMS が受信できる 携帯電話	AWS の本人確認用
スマホの認証アプリ	二段階認証用。 Google Authenticator (App Store / Play ストアで無料)を事前にインストール
パスワードマネージャー	この手順で作るパスワード・キーの保管用(1Password、Bitwarden、iCloud キーチェーン等。 キーをメールや LINE で平文送信するのは厳禁)
GitHub アカун ト	アプリのソースコードを受け取るため。 https://github.com で無料登録する

 **受け渡し①(この章を始める前):** GitHub アカウントを作成したら、**ユーザー名を開発者へ伝えて** リポジトリに招待してもらいます(1-4 のソースコード取得に必要です)。

1-2. 開発者から受け取るもの(事前にもらっておく)

受け取るもの	使う場所
GitHub リポジトリへの招待	1-4
移行用の読み取りアクセスキー(旧 AWS 環境用・2 つの文字列)	4 章
JWT シークレット 2 つ(staging 用 / production 用の長いランダム文字列)	5 章
メール送信元アドレス	5～6 章

1-3. Mac に入れるツール(この手順書で使うもの)

ツール	何をするもの？
ターミナル	Mac に最初から入っている「コマンド入力アプリ」。アプリケーション → ユーティリティ → ターミナル にある。この手順書のコマンドはすべてここに貼り付けて実行する
Homebrew	Mac 用の「ツールのインストーラー」。下のツールを入れるために最初に入れる
AWS CLI	ターミナルから AWS を操作するツール。データの引っ越しで使う
AWS SAM CLI	サーバー環境(Lambda / データベース / ストレージ一式)をコマンド 1 回で自動構築するツール
Node.js / esbuild	サーバーのプログラムを組み立てる(ビルドする)ためのツール
git / GitHub CLI	アプリのソースコードを GitHub から Mac に取得するツール
jq	データ引っ越しスクリプトが内部で使う、データ整形ツール

インストール手順

ターミナルを開き、以下を 1 ブロックずつコピーして貼り付け、Enter で実行します。

① Homebrew(5～10 分。途中で Mac のログインパスワードを求められたら入力。画面に文字は出ませんがそのまま Enter):

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

完了時に「Next steps」として `eval` で始まるコマンドが案内された場合は、それもコピーして実行してください。

② 残りのツールのインストール(5～10 分):

```
brew install awscli aws-sam-cli node git gh jq
npm install -g esbuild
```

③ インストール確認(それぞれバージョン番号が表示されれば成功):

```
aws --version && sam --version && node --version && git --version && jq --version && esbuild --version
```

1-4. アプリのソースコードを取得する

サーバー環境の「設計図」はソースコードの中に入っているため、Mac に取得しておきます。(開発者からの GitHub 招待を承諾してから実行してください)

```
gh auth login
```

- 質問には「GitHub.com」→「HTTPS」→「Login with a web browser」を選択(矢印キーで選んで Enter)
- 表示されたコードを控えて Enter → ブラウザが開くのでコードを入力して認証

```
cd ~  
gh repo clone yoshiydp/flexq-mobile
```

ホームフォルダに `flexq-mobile` フォルダができれば完了です。

第 2 章 AWS アカウントの作成

使うもの: ブラウザ、メール、クレジットカード、携帯電話、Google Authenticator

2-1. アカウント登録

1. ブラウザで <https://aws.amazon.com/jp/> を開き「無料でアカウントを作成」をクリック
2. メールアドレスと **AWS アカウント名**(例: flexq-ops)を入力し、届いた確認コードを入力
3. ルートユーザー(最上位アカウント)のパスワードを設定 → **パスワードマネージャーに保管**
4. 連絡先情報を入力(「個人」を選択で可)
5. クレジットカード情報を入力
6. 電話番号による本人確認(SMS でコードを受信して入力)
7. サポートプランは「**ベーシックサポート(無料)**」を選択
8. 完了後、<https://console.aws.amazon.com> にルートユーザーでログインできることを確認

2-2. 二段階認証(MFA)の設定 ※必須

アカウント乗っ取り防止のため、最初に必ず設定します。

1. ログイン後、画面右上のアカウント名をクリック → 「セキュリティ認証情報」
2. 「MFA を割り当てる」をクリック
3. デバイス名は任意(例: iphone)、**「認証アプリケーション」**を選択
4. 表示された QR コードを **Google Authenticator** で読み取り、アプリに表示される 6 桁の数字を 2 回分入力して登録

2-3. リージョン(データの置き場所)を東京にする

画面右上のリージョン表示(「バージニア北部」等になっている場合がある)をクリックし、「**アジアパシフィック(東京) ap-northeast-1**」を選択します。以後、AWS の画面作業はすべて「東京」になっていることを確認してから行ってください。

2-4. 請求アラートの設定 ※強く推奨

想定外の課金に気づけるよう、月 \$10 を超えたらメールが来るようにします。

1. 画面右上のアカウント名 → 「請求とコスト管理」
2. 左メニュー「予算(Budgets)」→ 「予算を作成」
3. 「テンプレートを使用」→ 「月次コスト予算」を選択
4. 予算額に 10 (米ドル)、通知先メールアドレスに自分のメールを入力して「予算を作成」

第 3 章 作業用ユーザーの作成

使うもの: ブラウザ、パスワードマネージャー

ルートユーザーは「金庫の鍵」なので日常作業には使いません。作業用のユーザー(IAM ユーザー)を 2 つ作ります。

3-1. 自分の作業用ユーザー

1. AWS の画面上部の検索バーに「IAM」と入力して開く
2. 左メニュー「ユーザー」→「ユーザーの作成」
3. ユーザー名: `flexq-admin`
4. 「AWS マネジメントコンソールへのユーザーアクセスを提供する」に**チェック**し、パスワードを設定
→ 保管
5. 許可の設定: 「ポリシーを直接アタッチする」→ 検索欄に `AdministratorAccess` と入力し、同名のポリシーにチェック → 「次へ」→ 「ユーザーの作成」
6. 作成したユーザー名をクリック → 「セキュリティ認証情報」タブ → 「**アクセスキーを作成**」
 - ユースケース: 「コマンドラインインターフェイス(CLI)」を選択し、確認にチェック
 - 表示された **アクセスキー ID** と **シークレットアクセスキー** をパスワードマネージャーに保管
(シークレットはこの画面を閉じると二度と表示されません)

3-2. 自動デプロイ用ユーザー(開発者に渡す)

GitHub の自動デプロイの仕組みが新しい AWS 環境へアクセスするために使います。

1. 同様に「ユーザーの作成」→ ユーザー名: `flexq-deploy`
2. コンソールアクセスのチェックは**入れない**
3. 許可: 同じく `AdministratorAccess`
4. アクセスキーを作成し、ID とシークレットを控える

 **受け渡し②(この章の終わり):** `flexq-deploy` のアクセスキー(ID とシークレットの 2 つ)を **パスワードマネージャーの共有機能で開発者へ渡します**。開発者はこれを GitHub の自動デプロイ設定に使います。3-1 の `flexq-admin` のキーは自分専用なので渡しません。

第 4 章 ターミナルと AWS をつなぐ

使うもの: ターミナル、AWS CLI、3-1 の自分のキー、開発者から受け取った旧環境のキー

データの引っ越しでは「旧環境(開発者の AWS)から読み取り」「新環境(自分の AWS)へ書き込み」の両方を行うため、2 つの接続設定(プロファイル)を登録します。

4-1. 新環境(自分のアカウント)の接続設定

```
aws configure --profile flexq
```

4 つ質問されるので、順に入力して Enter:

質問	入力する値
AWS Access Key ID	3-1 で作った自分のアクセスキー ID
AWS Secret Access Key	3-1 で作った自分のシークレットアクセスキー
Default region name	ap-northeast-1
Default output format	json

4-2. 旧環境(開発者のアカウント)の接続設定

開発者から受け取った**移行用の読み取りキー**を使い、同じ要領で登録します:

```
aws configure --profile lyrics-old
```

(region は同じく ap-northeast-1、format は json)

4-3. 接続テスト

```
aws sts get-caller-identity --profile flexq
aws sts get-caller-identity --profile lyrics-old
```

それぞれ Account に 12 桁の数字が表示されれば接続成功です。2 つの数字が**異なっていること**(= 別のアカウントにつながっていること)を確認してください。 lyrics-old 側は 140147588900 になっているはず

第 5 章 サーバー環境の立ち上げ

使うもの: ターミナル、AWS SAM CLI、開発者から受け取った JWT シークレット等

ソースコードに含まれる「設計図」(`api/template.yaml`)から、サーバー一式 (API プログラム約 25 個・データベース 5 つ・ファイル置き場)を自動構築します。 **staging(検証用)**と **production(本番用)**の 2 セットを作ります。

⚠ AWS の画面から手作業でデータベース等を作らないでください。設計図からの自動構築だけを使います。

i アプリの AI 機能(録音のクリーンアップ)に関する設定は、移行完了後に**開発者**が行います。この章の作業には含まれていないので、気にせず進めてください。

5-1. プログラムの組み立て(ビルド)

```
cd ~/flexq-mobile/api
sam build
```

数分待って `Build Succeeded` と表示されれば OK。

5-2. staging(検証用)環境の構築

`<...>` の 2 箇所を、開発者から受け取った値に置き換えてから実行します:

```
AWS_PROFILE=flexq sam deploy \
  --stack-name flexq-stg-api \
  --region ap-northeast-1 \
  --resolve-s3 \
  --capabilities CAPABILITY_IAM \
  --no-confirm-changeset \
  --parameter-overrides \
    JwtSecret="<staging 用 JWT シークレット>" \
    SenderEmail="<メール送信元アドレス>"
```

- 5~10 分かかります。 `Successfully created/updated stack - flexq-stg-api` と出れば成功
- 完了時に表示される **Outputs 欄**の **ApiUrl** (`https://~.amazonaws.com/v1` という URL)を**必ず控えてください**(この章の最後に開発者へ渡します)

5-3. production(本番用)環境の構築

同じ要領で、スタック名と JWT シークレットだけを変えて実行します:

```
AWS_PROFILE=flexq sam deploy \  
  --stack-name flexq-prod-api \  
  --region ap-northeast-1 \  
  --resolve-s3 \  
  --capabilities CAPABILITY_IAM \  
  --no-confirm-changeset \  
  --parameter-overrides \  
    JwtSecret="<production 用 JWT シークレット>" \  
    SenderEmail="<メール送信元アドレス>"
```

こちらも Outputs 欄の `ApiUrl` を控えます。

5-4. できあがりの確認

```
aws dynamodb list-tables --profile flexq  
aws s3 ls --profile flexq
```

- データベース(テーブル)が `flexq-stg-api-~` と `flexq-prod-api-~` で **5 つずつ、計 10 個**あること
- ファイル置き場(バケット)が `flexq-stg-api-trackaudiobucket-~` と `flexq-prod-api-trackaudiobucket-~` の **2 つ**あること

ブラウザでも確認できます: AWS の検索バーで「CloudFormation」→ スタック `flexq-stg-api` → 「リソース」タブに作られたもの一覧が表示されます。

 **受け渡し③(この章の終わり):** 5-2 と 5-3 で控えた **staging と production の `ApiUrl` (URL 2 つ)を、どちらがどちらか明記して開発者へ渡します。**第 8 章まで待たないでください — 開発者はこの URL を受け取った時点から、アプリ側の接続設定を並行して進められます。

第 6 章 メール送信の設定

使うもの: ブラウザ、メール

アプリの「パスワードを忘れた場合」のメール送信に Amazon SES というサービスを使います。

1. AWS の検索バーで「SES」と入力して開く(画面右上が「東京」であることを確認)
2. 左メニュー「ID」→「ID の作成」→「E メールアドレス」を選択し、開発者から指定された送信元アドレスを入力して作成
3. そのアドレス宛に AWS から確認メールが届くので、本文中のリンクをクリック (開発者管理のアドレスの場合は、開発者にクリックしてもらう)
4. SES の画面でステータスが「**検証済み**」になれば OK
5. **本番利用の申請**: 初期状態では検証済みアドレスにしかメールを送れない「サンドボックス」という制限付きです。SES ダッシュボードの「**本番アクセスをリクエスト**」から解除を申請します
 - リクエストの種類: E メール
 - ユースケースの説明(例): 「モバイルアプリのパスワードリセットメールの送信のみに使用します。送信は利用者の操作をきっかけとした 1 通ずつのメールのみで、一斉配信は行いません」
 - 通常 1 営業日程度で承認されます(承認待ちの間も他の章は進められます)

受け渡し④(2 回に分けて連絡):

1. 送信元アドレスが「検証済み」になった時点で、その旨を開発者へ連絡
2. 後日、本番アクセス承認のメールが届いた時点で、承認された旨を開発者へ連絡 (開発者はこの連絡を受けてから、パスワードリセットメールの動作確認を行います)

第 7 章 既存データの引っ越し

使うもの: ターミナル、AWS CLI、jq

旧環境(開発者の AWS)から新環境(自分の AWS)へ、**staging・production 両方**の「ファイル(音源・画像)」と「データベースの中身」をコピーします。

⚠ 実施タイミングは開発者と相談して決めてください。 引っ越し作業中にアプリが使われると、その分のデータがコピーから漏れます(アプリ利用を止めた状態で実施するのが確実です)。なお、この章の作業は**何度やり直しても安全**です(同じデータが上書きされるだけで、重複や破損はしません)。

7-1. ファイル(音源・画像)のコピー

一度 Mac にダウンロードしてから新環境へアップロードします。

① 旧・新それぞれのファイル置き場(バケット)の正確な名前を確認して控える:

```
aws s3 ls --profile lyrics-old
aws s3 ls --profile flexq
```

- 旧: `lyrics-mock-api-trackaudiobucket-~` (staging)と `lyrics-prod-api-trackaudiobucket-~` (production)
- 新: `flexq-stg-api-trackaudiobucket-~` と `flexq-prod-api-trackaudiobucket-~`

② staging 分をコピー(`<旧staging>` `<新staging>` は①で控えた実際の名前に置き換え):

```
mkdir -p ~/s3-migration/stg ~/s3-migration/prod
aws s3 sync s3://<旧staging> ~/s3-migration/stg --profile lyrics-old
aws s3 sync ~/s3-migration/stg s3://<新staging> --profile flexq
```

③ production 分をコピー:

```
aws s3 sync s3://<旧production> ~/s3-migration/prod --profile lyrics-old
aws s3 sync ~/s3-migration/prod s3://<新production> --profile flexq
```

④ 検証 — ファイル数と合計サイズが新旧で一致することを確認(staging・production それぞれ):

```
aws s3 ls s3://<旧バケット名> --recursive --summarize --profile lyrics-old | tail -2
aws s3 ls s3://<新バケット名> --recursive --summarize --profile flexq | tail -2
```

最後に表示される `Total Objects:` (個数)と `Total Size:` (サイズ)が新旧で同じなら OK。

7-2. データベースの中身のコピー

コピー用のスクリプト(自動処理プログラム)を Mac に保存して実行します。

- ① スクリプトの保存(下のブロック全体を丸ごとコピーしてターミナルに貼り付け、Enter):

```

mkdir -p ~/s3-migration
cat > ~/s3-migration/migrate-dynamodb.sh <<'SCRIPT'
#!/bin/bash
set -euo pipefail

OLD_PROFILE=lyrics-old
NEW_PROFILE=flexq
REGION=ap-northeast-1
WORKDIR=$(mktemp -d)

STACK_PAIRS=(
    "lyrics-mock-api:flexq-stg-api"
    "lyrics-prod-api:flexq-prod-api"
)

for PAIR in "${STACK_PAIRS[@]"; do
    OLD_STACK="${PAIR%%:*}"
    NEW_STACK="${PAIR##*:}"
    echo "##### ${OLD_STACK} -> ${NEW_STACK}"

    for RESOURCE in UsersTable ProjectsTable TracksTable RecordsTable MemosTable; do
        OLD_TABLE=$(aws dynamodb list-tables --profile "$OLD_PROFILE" --region "$REGION" \
            --output text --query "TableNames[?starts_with(@, '${OLD_STACK}-${RESOURCE}-')]" |
[0])
        NEW_TABLE=$(aws dynamodb list-tables --profile "$NEW_PROFILE" --region "$REGION" \
            --output text --query "TableNames[?starts_with(@, '${NEW_STACK}-${RESOURCE}-')]" |
[0])

        if [ "$OLD_TABLE" = "None" ] || [ "$NEW_TABLE" = "None" ]; then
            echo "ERROR: ${RESOURCE} のテーブルが見つかりません (old=$OLD_TABLE new=$NEW_TABLE)"; ex
it 1
        fi

        echo "=== ${RESOURCE}: ${OLD_TABLE} -> ${NEW_TABLE}"

        SCAN_FILE="$WORKDIR/${OLD_STACK}-${RESOURCE}.json"
        echo '[]' > "$SCAN_FILE"
        NEXT_TOKEN=""
        while ;; do
            if [ -z "$NEXT_TOKEN" ]; then
                RESP=$(aws dynamodb scan --table-name "$OLD_TABLE" --profile "$OLD_PROFILE" --regi
on "$REGION" --output json)
            else
                RESP=$(aws dynamodb scan --table-name "$OLD_TABLE" --profile "$OLD_PROFILE" --regi
on "$REGION" --output json \
                --exclusive-start-key "$NEXT_TOKEN")
            fi
        done
    done
done

```

```

fi
echo "$RESP" | jq '.Items' > "$WORKDIR/page.json"
jq -s '.[0] + .[1]' "$SCAN_FILE" "$WORKDIR/page.json" > "$WORKDIR/merged.json"
mv "$WORKDIR/merged.json" "$SCAN_FILE"
NEXT_TOKEN=$(echo "$RESP" | jq -c '.LastEvaluatedKey // empty')
[ -z "$NEXT_TOKEN" ] && break
done

COUNT=$(jq 'length' "$SCAN_FILE")
echo "    ${COUNT} 件を取得。書き込み中..."

i=0
while [ "$i" -lt "$COUNT" ]; do
    jq -c --argjson i "$i" "{\"${NEW_TABLE}\": [.\${i}:\${i+25}] | {PutRequest: {Item:
.}}}" \
        "$SCAN_FILE" > "$WORKDIR/batch.json"
    aws dynamodb batch-write-item --request-items "file://$WORKDIR/batch.json" \
        --profile "$NEW_PROFILE" --region "$REGION" --output json \
        | jq -e '.UnprocessedItems == {}' > /dev/null \
        || { echo "ERROR: 未処理アイテムがあります。再実行してください"; exit 1; }
    i=$((i + 25))
done
echo "    完了"
done
done

echo "##### 全テーブルの移行が完了しました"
rm -rf "$WORKDIR"
SCRIPT
chmod +x ~/s3-migration/migrate-dynamodb.sh

```

② 実行:

```
~/s3-migration/migrate-dynamodb.sh
```

各テーブルごとに「N 件を取得。書き込み中... 完了」と表示され、最後に「**全テーブルの移行が完了しました**」と出れば成功です。途中でエラーが出た場合は、そのままもう一度実行して構いません(直らなければ開発者へ相談)。

③ 検証 — ブラウザで AWS の検索バーから「DynamoDB」→「テーブル」→ 任意のテーブル名をクリック → 「**テーブルアイテムの探索**」で、実際のデータが入っていることを目で確認します。



受け渡し⑤(この章の終わり): データ検証の結果を開発者へ報告します。

- 7-1 ④のファイル数・合計サイズが新旧で一致したか(staging / production それぞれ)

- 7-2 ②が「全テーブルの移行が完了しました」まで進んだか、③の目視確認ができたか
- 一致しなかった・エラーが出た場合はその内容(スクリーンショット)

この報告をもって運営者側の移行作業は完了です。

第 8 章 完了報告(受け渡しの最終確認)

各章の 📁 マークで渡してきたものが、すべて開発者へ渡っているかを最終確認し、「移行作業が完了した」ことを開発者へ連絡します。ここから先(アプリの接続先切り替え・最終動作確認)は開発者の作業です。

済	渡したもの	渡したタイミング
☐	① GitHub のユーザー名	第 1 章の前
☐	② flexq-deploy のアクセスキー(ID とシークレット)	第 3 章の終わり(3-2)
☐	③ staging / production の ApiUrl (URL 2 つ)	第 5 章の終わり(5-4)
☐	④ SES の検証完了・本番アクセス承認の連絡	第 6 章(承認メール到着時)
☐	⑤ データ検証結果の報告	第 7 章の終わり

未提出のものがあれば、この時点でまとめて渡してください。

後片付け(開発者の最終確認が済んでから):

- 開発者から「切り替え完了・旧環境も削除した」と連絡が来たら、Mac の ~/s3-migration フォルダを削除して構いません(引っ越し用の一時コピーが入っています):

```
rm -rf ~/s3-migration
```

第 9 章 困ったときは

症状	対処
コマンドで <code>command not found</code> と出る	第 1 章のインストールが済んでいるか確認。ターミナルを一度閉じて開き直す と直すこともある
<code>Unable to locate credentials</code> と出る	第 4 章の接続設定を確認。デプロイ時はコマンド先頭に <code>AWS_PROFILE=flexq</code> が付いているか確認
<code>sam build</code> が失敗する	<code>npm install -g esbuild</code> を再実行してから <code>sam build</code> をやり直す
<code>aws s3 sync</code> で権限エラー	旧側に <code>--profile lyrics-old</code> 、新側に <code>--profile flexq</code> が付いている か確認
デプロイが <code>ROLLBACK</code> と表示 され失敗	ブラウザで CloudFormation → 該当スタック → 「イベント」タブの赤い行 をスクリーンショットして開発者へ相談
確認メールが届かない	迷惑メールフォルダを確認。SES の ID を作り直して再送も可
上記以外	エラーが出た画面(ターミナルは直前のコマンドから)をスクリーンショットして 開発者へ相談